

SALESTORM: API & Event Specification

Team: [Binary Minds]

Machine-readable version: `openapi.yaml` (OpenAPI 3.0, import into Swagger Editor or Postman)

1. Conventions

Topic	Rule
Base URL / version	<code>https://api.salestorm.example/api/v1</code> . Breaking changes go to <code>/v2</code>
Transport	HTTPS only (TLS 1.2+), JSON bodies
Authentication	OAuth2/OIDC bearer JWT validated at the API Gateway. Admin routes need the ADMIN role
Authorization	Ownership check on every customer resource (a customer sees only their own reservations, orders and payments)
Idempotency	Idempotency-Key header (UUID) is required on POST <code>/reservations</code> , POST <code>/checkout</code> , POST <code>/orders/{id}/cancel</code> and admin POSTs
Tracing	traceparent in, X-Request-Id out, on every call
Errors	RFC 7807 application/problem+json with a stable machine-readable code
Rate limiting	Per user and per IP at the gateway. Returns 429 with Retry-After
Pagination	Cursor based (cursor, limit max 50)
Timestamps / money	ISO-8601 UTC. Amounts as decimals with a currency field

2. Endpoint Summary

Method	Path	Purpose	Auth	Success	Main errors
GET	<code>/sales/{saleId}</code>	Sale details and window (CDN cached)	Public	200	404
GET	<code>/products/{productId}/stock-status</code>	AVAILABLE / LOW_STOCK / SOLD_OUT (cached)	Public	200	404
GET	<code>/cart</code>	View cart	Customer	200	401
POST	<code>/cart/items</code>	Add item	Customer	201	400, 401
DELETE	<code>/cart/items/{itemId}</code>	Remove item	Customer	204	401, 404
POST	<code>/reservations</code>	Buy Now: reserve stock	Customer	201 (200 on replay)	400, 409, 422, 429, 503
GET	<code>/reservations/{id}</code>	Status, and the orderId once it exists	Customer	200	404
DELETE	<code>/reservations/{id}</code>	Release my reservation (repeat-safe)	Customer	204	404, 409
POST	<code>/checkout</code>	Pay for a reservation	Customer	200 PAID / 202 PROCESSING	402, 409, 410, 422, 503
GET	<code>/payments/{id}</code>	Payment status (poll after 202)	Customer	200	404
GET	<code>/orders</code>	My orders (paged)	Customer	200	401
GET	<code>/orders/{id}</code>	One order	Customer	200	404
POST	<code>/orders/{id}/cancel</code>	Cancel before processing, refund async	Customer	202	409

Method	Path	Purpose	Auth	Success	Main errors
GET	/orders/{id}/tracking	Shipment and delivery tracking	Customer	200	404
POST	/webhooks/payment-gateway	Gateway result callback	HMAC signature	200	400, 401
GET	/admin/sales/{saleId}/metrics	Live sale metrics	Admin	200	403
POST	/admin/reconciliation/run	Trigger reconciliation	Admin	202	403
POST	/admin/inventory/{productId}/rebuild-gate	Reset Redis counter from the DB	Admin	200	403
POST	/admin/dead-letters/{messageId}/replay	Replay a dead-lettered message	Admin	202	403, 404

Internal (service to service, mTLS, not exposed at the gateway): Checkout → Inventory: markPaymentPending(reservationId), Checkout → Payment: pay(reservationId, amount, idemKey), Reconciliation → Payment: queryStatus / refund.

3. Key Endpoints in Detail

3.1 POST /reservations (Buy Now)

Request

```
POST /api/v1/reservations
Authorization: Bearer <jwt>
Idempotency-Key: 7f3c1e52-9c1a-4b8e-a1c4-2d9e5b7a6f10
Content-Type: application/json

{ "saleId": 501, "productId": 9001, "quantity": 1 }
```

201 Created

```
{
  "reservationId": "b1d5c8a2-3f0e-4c57-9a2b-6e1f0d3c7a44",
  "productId": 9001,
  "status": "RESERVED",
  "expiresAt": "2026-10-05T10:20:00Z",
  "orderId": null
}
```

409 Out of stock

```
{
  "type": "https://api.salestorm.example/errors/out-of-stock",
  "title": "Out of stock",
  "status": 409,
  "code": "OUT_OF_STOCK",
  "detail": "All units have been reserved or sold.",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Case	Response
Won a unit	201 + reservation
Duplicate request (same key and body)	200 + original reservation, header Idempotent-Replayed: true
Same key, different body	422 IDEMPOTENCY_KEY_REUSED
Same key still being processed	409 REQUEST_IN_PROGRESS + Retry-After
New key but customer already holds a unit	409 PER_CUSTOMER_LIMIT
Gate or DB says no stock	409 OUT_OF_STOCK

Case	Response
Sale not open	409 SALE_NOT_ACTIVE
Overload / waiting room active	429 or 503 + Retry-After

3.2 POST /checkout

Request

```
{ "reservationId": "b1d5c8a2-3f0e-4c57-9a2b-6e1f0d3c7a44", "paymentMethodToken": "tok_9d8s7a",
  "couponCode": null }
```

(plus Idempotency-Key header. Raw card data never reaches SALESTORM, only the gateway's token.)

Outcome	Status	Body / behaviour
Payment succeeded	200	{ "status": "PAID", "message": "Payment received, confirming your order" }. Order appears shortly after (async)
Gateway timed out	202	{ "status": "PROCESSING" }. Client polls GET /payments/{id} or GET /reservations/{id}. No re-charge
Declined	402 PAYMENT_DECLINED	Reservation released, customer informed
Reservation expired	410 RESERVATION_EXPIRED	No charge attempted
Reservation not in a payable state	409 INVALID_STATE	
Duplicate request	200/202	Stored original response, Idempotent-Replayed: true
Payment dependency down (circuit open)	503 + Retry-After	Reservation stays valid until expiry. Safe to retry with the same key

3.3 POST /webhooks/payment-gateway

- Verify X-Gateway-Signature (HMAC-SHA256 of the raw body) and X-Gateway-Timestamp (reject if older than 5 minutes).
- De-duplicate by the gateway eventId.
- Apply through the same idempotent payment update as the synchronous path and reconciliation, then return 200 quickly.

3.4 POST /orders/{id}/cancel

Allowed in CREATED, PAYMENT_PENDING, CONFIRMED and PROCESSING. Returns 202 because the refund is asynchronous (OrderCancelled event → Payment issues the refund). 409 INVALID_STATE once shipped.

4. Error Code Catalogue

code	HTTP	Meaning	Client action
VALIDATION_ERROR	400	Bad or missing field	Fix request
IDEMPOTENCY_KEY_MISSING	400	Required header absent	Add a UUID key
UNAUTHENTICATED	401	Missing, expired or invalid token or signature	Log in again
PAYMENT_DECLINED	402	Gateway declined	Try another method
FORBIDDEN	403	Role or ownership check failed	None
NOT_FOUND	404	Resource does not exist or is not yours	None
OUT_OF_STOCK	409	No units left	Show sold-out

code	HTTP	Meaning	Client action
SALE_NOT_ACTIVE	409	Outside the sale window	Wait
PER_CUSTOMER_LIMIT	409	Already holds a unit	Go to checkout
INVALID_STATE	409	Action not allowed in the current state	Refresh status
REQUEST_IN_PROGRESS	409	Same key still processing	Retry after delay
RESERVATION_EXPIRED	410	Reservation window passed	Buy Now again if stock remains
IDEMPOTENCY_KEY_REUSED	422	Same key, different payload	Use a new key
RATE_LIMITED	429	Too many requests	Back off for Retry-After
SERVICE_UNAVAILABLE	503	Overload shedding or open circuit	Retry with the same key

5. Idempotency Design

Aspect	Rule
Scope	Key is unique per customer and endpoint
Storage	First in Redis (SET NX, fast path), then the database unique constraint as the final guard (inventory_reservation.idempotency_key, payment.idempotency_key)
Retention	24 hours (longer than the reservation window)
Same key, same body	Returns the stored original response (Idempotent-Replayed: true)
Same key, different body	422 IDEMPOTENCY_KEY_REUSED
In flight	409 REQUEST_IN_PROGRESS rather than running twice
Payment to gateway	The gateway call carries the transaction_ref as its own idempotency key, so a retry cannot charge twice
Messages	Consumers record (consumer, eventId) in processed_message inside the same transaction as their change

6. Event & Message Design

6.1 Envelope (all events)

```
{
  "eventId": "0e8a6c52-1f4d-4d6e-8b1a-3c9d7e2f5a10",
  "eventType": "PaymentSucceeded",
  "version": 1,
  "occurredAt": "2026-10-05T10:12:31Z",
  "producer": "payment-service",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "aggregateId": "b1d5c8a2-3f0e-4c57-9a2b-6e1f0d3c7a44",
  "payload": { "paymentId": "...", "reservationId": "...", "amount": 4999.00, "currency": "INR",
  "transactionRef": "TXN-..." }
}
```

Property	Rule
Delivery	At-least-once, so consumers must be idempotent
Ordering	Partition key = reservationId, which keeps events for one purchase in order
Publishing	Producers write the event to an outbox table in the same transaction as the state change. The Outbox Relay publishes it
Failures	Retry with exponential backoff, then <topic>.dlq with an alert. Replay through the admin endpoint
Schema evolution	Add fields only. Breaking changes bump version

6.2 Event Catalogue (ownership and consumers)

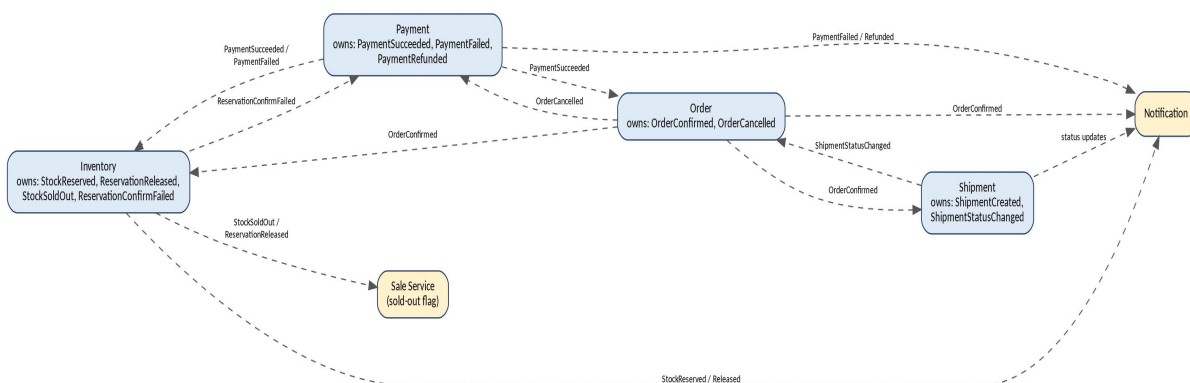
Event	Owner (producer)	Topic	Consumers and what each does
StockReserved	Inventory	inventory.events	Notification: “reserved, pay within 10 minutes”. Analytics: conversion funnel
ReservationReleased	Inventory	inventory.events	Notification: expiry or failure message. Sale Service: clear the sold-out flag if stock returned
StockSoldOut	Inventory	inventory.events	Sale Service: set the cached sold-out flag. Gateway/CDN: purge so rejections happen at the edge
ReservationConfirmed	Inventory	inventory.events	Analytics, Observability
ReservationConfirmFailed	Inventory	inventory.events	Payment: refund (payment arrived after the reservation was released)
PaymentSucceeded	Payment	payment.events	Order: create order. Inventory: reservation to CONFIRMED
PaymentFailed	Payment	payment.events	Inventory: release reservation. Notification: failure message
PaymentRefunded	Payment	payment.events	Order: mark cancelled/refunded. Notification: refund message
OrderConfirmed	Order	order.events	Inventory: reservation to SOLD. Shipment: create shipment. Notification: confirmation
OrderCancelled	Order	order.events	Payment: refund. Notification
ShipmentCreated / ShipmentStatusChanged	Shipment	shipment.events	Order: advance to PROCESSING, SHIPPED, OUT_FOR_DELIVERY, DELIVERED. Notification: tracking updates

Rule: each event has exactly **one owner**. Only the owner may change the underlying data, and consumers react but never write to the owner’s tables.

6.3 Commands (requests for one service to act)

Command	From → To	Style
ReserveStock	Checkout → Inventory	Sync API
MarkPaymentPending	Checkout → Inventory	Sync, internal
InitiatePayment	Checkout → Payment	Sync with timeout
RefundPayment	Order or Reconciliation → Payment	Async (queue)
ReleaseReservation	Expiry Worker or customer → Inventory	Internal job or DELETE
ReplayDeadLetter	Admin → Broker	Admin API

6.4 Event Flow



Event flow between services (dashed arrows are asynchronous events)

7. Security Notes for the API

- JWT is validated at the gateway (signature, expiry, audience). Services re-check ownership.
 - Reservation and checkout routes have the strictest rate limits (for example 5 reservation attempts per minute per user) and are covered by WAF bot rules.
 - The webhook route is the only unauthenticated-by-JWT route. It relies on the HMAC signature, a timestamp window and an IP allow-list.
 - Responses never include exact stock counts, only the coarse stock-status.
 - Payment data: only gateway-issued tokens are accepted. Nothing sensitive is logged. Audit rows record actor, action and trace id.
-